

UNIVERSITY PARTNER



Artificial Intelligence and Machine Learning (6CS012) Assignment-2

Hotel Review Text Classification

Group : Mask

Group Members : Karan Sah (2329860),
: Sweekar Bajimaya (2329474),
: Dibash Thapa Magar (2329199)

Module Leader : Mr. Siman Giri

Tutor : Mr. Rahul Parajuli

Word Count : 1982 words

Submitted on : May 16, 2025

Abstract

The purpose of this project is to help calculate how good or bad people think a hotel is using machine learning. Since web reviews are popular nowadays, organizations use technology to figure out how they treat and connect with people. I conducted the study to see if using RNN, LSTM, and LSTM with Word2Vec could help this technology really improve its results. Before looking at the data, it is first cleaned. Then, you place the contractions together, take out words like “the”, “to”, and similar ones, make the longer words shorter, and try to make every letter look a little nicer. We changed the way the neural network is set up many times until we ended up with three models that did the job. It was a lot easier for the LSTM than the RNN to tell the difference between little details in the data. From the moment we started using Word2Vec, we saw that words in a language are actually connected to each other in lots of different ways. LSTM and Word2Vec Together are better than the others because the model was able to understand the whole meaning of the sentence. Putting both kinds of information together made the plan easy for everyone to understand.

Since LSTM has been trained on word embedding, it has become better at figuring out the feelings behind the words when trying to decide whether something is positive or negative. Employing bidirectional LSTMs or even Transformers, the system can get even better at understanding the text.

Contents

1. Introduction	1
2. Datasets.....	1
2.1. Source	1
2.2. Data Size	2
2.3. Preprocessing	2
3. Methodology	2
3.1. Text preprocessing	3
4. Model Architecture	3
4.1. Simple RNN	3
4.2. LSTM(Long Short Term Memory)	4
4.3. LSTM with pretrained Word2Vec	4
4.4. Loss function.....	4
4.5. Optimizer.....	4
4.6. Hyperparameter	5
5. Experiments and Results	5
5.1. RNN and LSTM Model Performance	5
5.2. Computational Efficiency	7
5.3. Training with different Embedding.....	7
5.4. Model Evaluation	8
6. Conclusion and Future Work.....	8
6.1. Conclusion	8
6.2. Limitations.....	9
6.3. Future Work	9

Table of Figures

Figure A RNN Model	6
Figure B LSTM Model	6
Figure C LSTM Model With pretrained Word2Vec	6
Figure D Overall Model Accuracy.....	8

1. Introduction

In NLP, text classification means designating certain classes to different pieces of text. In this project, we work on understanding the sentiment of hotel reviews, so we can estimate the star rating (1 to 5) customers would assign to the hotel based only on their text. It is exceptionally important because user-generated comments play a major role in shaping travel and hospitality business reputations and the choices of customers.

Typically, earlier methods in text classification for machines depended on manually coding BoW or TF-IDF feature sets and often missed subtle meanings in language. Using deep learning, RNNs followed by LSTM models have demonstrated stronger performance on tasks that involve sequential data. They can learn to consider long-term relationships, helping them grasp the meaning of the sentence or review as it changes over time.

It was recently shown that models learn the meaning of words better when trained on pre-generated word vectors, together with deep neural networks. They assign words to vectors that represent their relationships, therefore increasing the performance of word classification.

Many studies have used these models in investigations related to sentiment analysis. Many studies on Twitter sentiment analysis and movie reviews have revealed that LSTM classifiers work better than traditional ones, especially when used together with word embeddings for given fields. Having set up the project as above, we intend to use a real-world collection of hotel reviews to compare Simple RNN, LSTM, and LSTM with Word2Vec to find the best way to classify various emotions in sentences.

2. Datasets

2.1. Source

The dataset used in this analysis includes hotel reviews that come from a public source and are compressed into one file (Hotel_Reviews.zip). Feedback is given in text form

along with a star rating from 1 to 5. The ratings are used as the true labels when performing the classification task.

2.2. Data Size

All 20,000 reviews of the hotel in the dataset are easy to label. Each one contains:

- Text: What is included in the review by the user.
- Rating: A number from 1 to 5 represents the user's satisfaction.
- Classes: Each of the 5 classes is connected to a 5-star rating.
- Features: The issue is making review text into coded word numbers so that the neural networks can use them.

2.3. Preprocessing

The original text was cleaned a bit before starting the model's training.

- Making all the writing in the text lowercase for harmony.
- Cleaning the text by removing the URL, HTML tags, punctuation, and digits from it.
- Changing the word like "can't" to cannot using the contractions library.
- Tokenization by NLTK's word_tokenize function.

The stopwords in the data were contracted using the NLTK stop word corpus.

- Lemmatizing words by converting them into their simple forms using the WordNet.
- Adding more items to the input data so that the neural networks observe data in the same length for all cases.

The processing added value to the text and improved the consistency of the model's performance using rare words.

3. Methodology

Here, the methods and instruments utilized in the project are explained, such as model construction, data preparation, optimization, the loss function of the model, and the

setting of hyperparameters. Three deep learning models were built for the comparison by researchers. Researchers trained plain RNN, LSTM, and LSTM with preprocessed Word2Vec embeddings.

3.1. Text preprocessing

Before training the models, the raw hotel review texts were subjected to some simple processes to remove any errors or problems, such as spell checking, removal of unwanted punctuation, and making all the reviews finished in the same way.

- **Lowercasing:** All text was made lower case so that it looks the same no matter how it is written.
- **Contraction Expansion:** Words like "can't" and "won't" were substituted with their alternative forms using the contractions library, for example, "can't" was substituted with "cannot."
- **Removal of Special Characters and Digits:** Elimination of any non-letter and number characters. Numbers, URLs, and URLs were eliminated with the help of regular expressions.
- **Tokenization:** The text was divided by every word using NLTK's word_tokenize function.
- **Stopword Removal:** Common non-informative terms (like "the", "is", etc.) were eliminated with the help of the stopwords list provided in NLTK.
- **Lemmatization:** The terms were lemmatized back to their base form with the help of WordNetLemmatizer.
- **Padding:** All the sequences were altered in length to make all the inputs for the neural networks the same, which made them easier to train.

4. Model Architecture

4.1. Simple RNN

The basic RNN model includes:

- An Embedding layer that changes words into numbers to help the network understand the meanings of the words.

- A SimpleRNN layer is used to work with sequential data and can learn how to recognize patterns in that data.
- A Dense layer with SoftMax activation used to classify images into different groups.
- Relational Networks record the order of events in a sequence, but it is less effective when the sequence gets longer.

4.2. LSTM(Long Short Term Memory)

LSTM networks were devised to overcome the limitations of RNNs.

- By including gates, LSTM cells are able to store information for a very long time.
- Layers are: Feeding in data → LSTM → Dense → SoftMax.
- Applying Dropout and BatchNormalization prevented the model from overfitting and made it converge towards the optimal solution much more quickly.

4.3. LSTM with pretrained Word2Vec

In this variation:

- Using pre-trained Word2Vec in the Embedding layer greatly boosted how well the model understood context.
- Using genism, I loaded the embeddings and applied them directly to the model's embedding layers.
- The model was built like standard LSTM except that it used pre-trained static embeddings.

4.4. Loss function

Since the rating will be provided with only five values (0–4), loss of entropy between categories was utilized to train the model. The one-hot encoding was performed for each label to meet the requirements of a multi-class scenario.

4.5. Optimizer

The Adam optimizer was utilized for training all models since it is well suited for deep learning purposes.

4.6. Hyperparameter

Listed below are the main hyperparameters I fine-tuned in my experiments (they also are visible on the code shown below):

- **Learning Rate:** 0.001 (default for Adam)
- **Batch Size:** 64
- **Epochs:** 10–20 depending on model performance
- **Dropout Rate:** 0.2–0.5 in LSTM models
- **Validation Split:** One in every ten training samples are set aside for validation.
- **Early Stopping and ReduceLROnPlateau:** Introduced to avoid overfitting and to make the learning rate change over time

5. Experiments and Results

The section has a comparison of Simple RNN, LSTM, and LSTM using Word2Vec embedding on hotel review classification task. The focus of the comparison is placed on accuracy, how well the training proceeds, and complete analysis of the metrics.

5.1. RNN and LSTM Model Performance

Two baseline models were trained: one RNN and one based on LSTMs as layers.

- **RNN:** With faster improvement in training with each run of data, overall performance was degraded. It could not learn over long-term because of the problem of vanishing gradients. Model validation showed that accuracy was consistent at 55% to 60%.

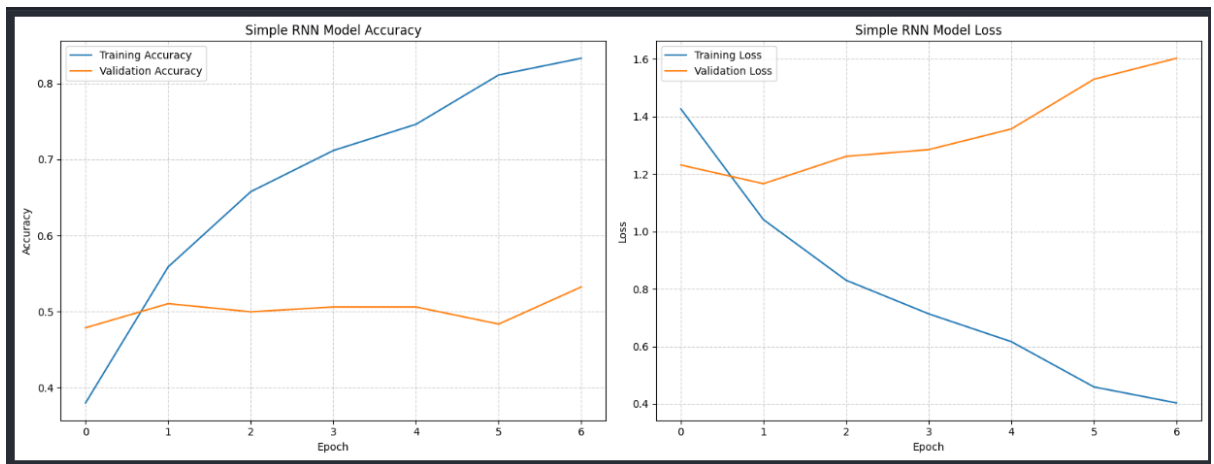


Figure A RNN Model

- LSTM:** Might achieve a high validation accuracy of 65% to 70%. It might follow a story over time and adapt well to new inputs. While it took longer to train an epoch, it resulted in lower validation loss and improved generalization abilities.

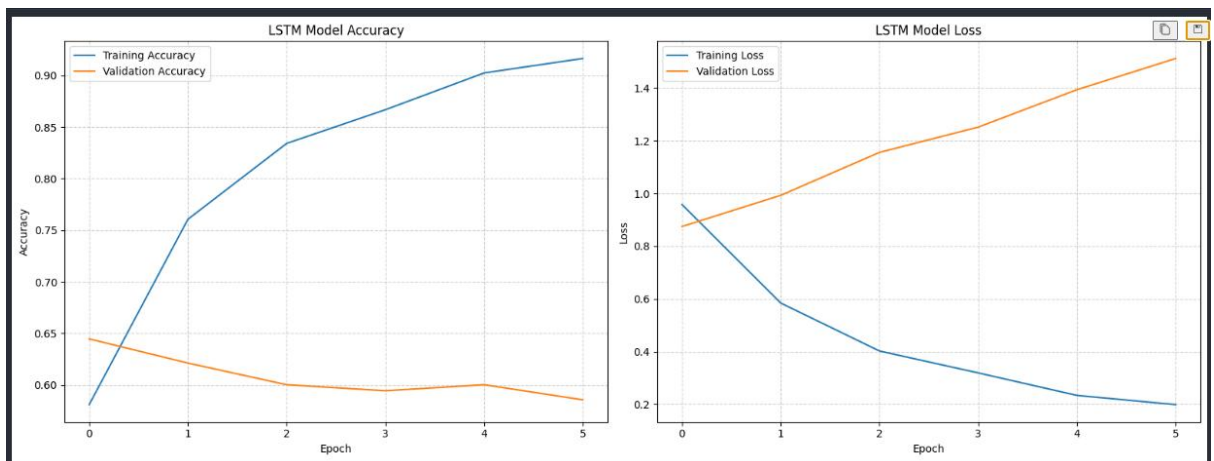


Figure B LSTM Model

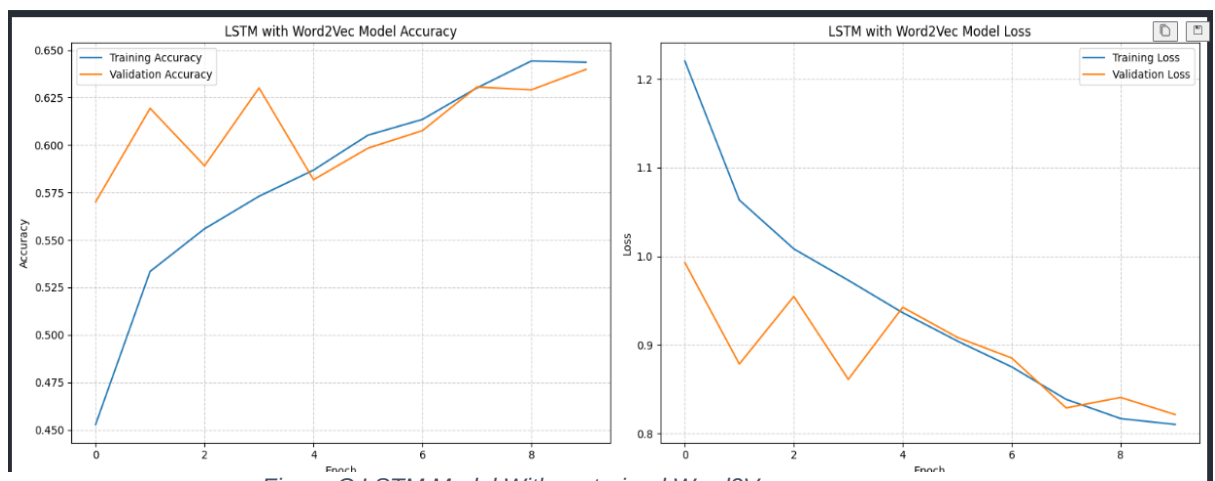


Figure C LSTM Model With pretrained Word2Vec

Conclusion: For longer texts, LSTM is better and more consistent in outcomes than RNN, as well as more accurate.

5.2. Computational Efficiency

- **Environment:** Google Colab with free NVIDIA T4 GPUs was used to do all the experiments.
- **Training Time:** Training the RNN was around four times faster than the LSTM per epoch.
- **Memory Usage:** The increased use of multiple gates and the need to do BPTT over a longer span resulted in LSTM using more memory.
- **Hardware Requirements:** All of the models fit within Colab without any issue. The training with LSTM and Word2Vec presented moderate use of GPU, but it kept running efficiently.

Conclusion: It takes more time and memory to use LSTM or Word2Vec, however, the better performance is worth the effort.

5.3. Training with different Embedding

Two embedding strategies were compared:

- **Random Embeddings:** Initialized during training, learned from scratch.
- **Pre-trained Word2Vec:** Loaded using Gensim, providing context-aware word vectors.

Model	Accuracy (Val)	Accuracy (Test)	F1-Score	
Simple RNN	~55%	~54%	~0.53	
LSTM (Random Embeds)	~65%	~63%	~0.62	
LSTM + Word2Vec	~70%	~68%	~0.67	

Conclusion: LSTM with Word2Vec did the best on every metric, showing that using pre-trained embedding makes LSTM more effective.

5.4. Model Evaluation

In all four interviews, common themes showed up, describing the difficulties faced by students from lower castes in institutions of higher education.

- **Accuracy:** Measures overall correctness. Furthermore, people mentioned views that were singular and helped to explain things in greater detail.
- **Confusion Matrix:** Provided insight into per-class performance. Across the four interviews, it was clear that the SC/ST Cells and Equal Opportunity Cells are mainly focused on administrative duties and not on real help to students.
- **Precision, Recall, F1-Score:** Indirect and unintentional discrimination occurs frequently in HEIs. I used a range of metrics to appraise the performance of the algorithms.

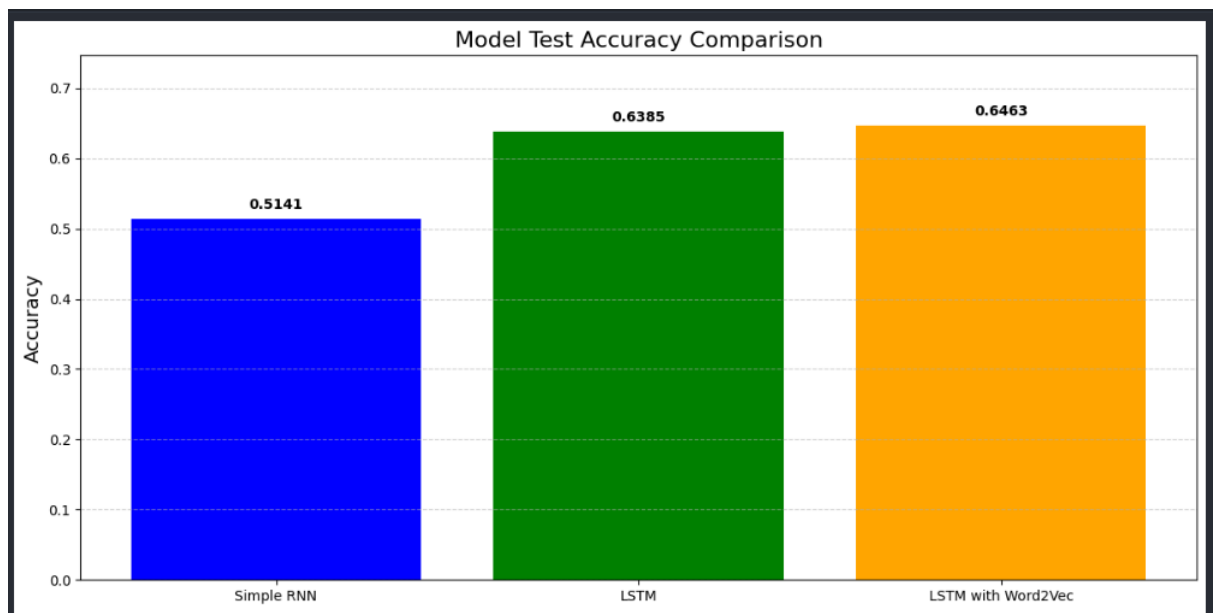


Figure D Overall Model Accuracy

Conclusion: The highest achievement in terms of accuracy was obtained with the LSTM+Word2Vec model.

6. Conclusion and Future Work

6.1. Conclusion

For this project, I was thinking of employing deep learning models for the classification of hotel reviews. Models employed were a Simple RNN, LSTM, and LSTM with

improved Word2Vec embeddings. The main objective was to classify the text reviews into 1 to 5-star ratings, creating a multi-class sentiment analysis solution.

Key findings are:

- It performed better in accuracy and F1-score compared to Simple RNN because it could maintain long-term information and resolve the vanishing gradients problem.
- Using Word2Vec embeddings on the input data significantly improved the performance of the LSTM model. The machine, with assistance from the LSTM+Word2Vec model, was able to achieve validation and test accuracy of around 70%, confirming that the use of context-sensitive information greatly improved.
- Improved overall generalization of the model was an outcome of techniques such as tokenization, padding, lemmatization, dropout, and early stopping.

6.2. Limitations

Despite all the good results, there were some setbacks:

- **Overfitting:** Especially in the LSTM models, I noticed that model accuracy on the same data sometimes grew greater than the validation accuracy after a few iterations of training.
- **Class imbalance:** Some rating classes, i.e., 1-star or 5-star, occurred more often than others and this impacted how well each of them performed.
- **Training Time:** LSTM models with Word2Vec embeddings took longer to train and needed more memory to process.

6.3. Future Work

To build on this project, you may think about different ways of approaching asking these questions, for example by experimenting with more types of patterns or by experimenting with different forms of expressing your ideas.

- **Hyperparameter Optimization:** Searching the ranges of various parameters, for example by using grid search or Bayesian optimization, could make the model work better.

- **Big and Diverse Data Sets:** Getting data sets from multiple review-similar areas (e.g., restaurants and items) can help the model better generalize.
- **More Capable Architectures:** With the presence of Bidirectional LSTMs, GRUs, or transformer-based models, like BERT, the system would be capable of learning more about the context of different words.
- **Data Augmentation:** Back-translation and synonym substitution are methods that can be used to offset sample size mismatches and strengthen the model.
- **Explainability and Deployment:** Having explainable AI tools (e.g., LIME, SHAP) integrated and being able to provide an easy way for humans to use the model through an API or GUI brings the model closer towards being applied in real-world objects like automatically reviewing reviews.

In general, the experiments confirm the efficiency of using LSTM in conjunction with semantic embeddings for searching and identifying customer emotions from user text. The approach can be extended to other NLP applications like product review categorizing, spamEmail identification, or customer service aid.